Racing Line Optimization Using NLP

Nimmi M Giji, CSE S4 B1 57

April 28, 2024

1 Introduction

In one general form, the nonlinear programming problem is to find $x = (x_1, x_2, \ldots, x_n)$ so as to

$$\begin{aligned} &\text{Maximize} \quad f(x) \\ &\text{subject to} \\ &\qquad g_i(x) \leq 0 \quad \text{for } i = 1, 2, \ldots, m \\ &\qquad x \geq 0 \\ &\qquad b_i \leq x_i \leq b_i' \quad \text{for } i = 1, 2, \ldots, n, \end{aligned}$$

where $f(x)$ and the $g_i(x)$ are given functions of the n decision variables.

In racing, drivers strive to find the fastest route around a track, also known as the racing line. There are countless possibilities for racing lines, but the optimal one minimizes lap time. This ideal line considers track conditions and adapts accordingly. While straight sections offer little variation, corners are where racing lines truly make a difference. Here, drivers must balance speed with the length of the line they take. When focusing on a single corner, the optimal racing line is the one that minimizes the time spent in the corner while maximizing the vehicle's speed. Opting for the shortest radius path minimizes the distance travelled around the corner. However, adopting a wider curve, thus reducing curvature, allows for higher speeds, potentially offsetting the extra distance covered. When considering the entire track, the optimal racing line aims to minimize total time while maximizing speed throughout the track. A smoother, less curvy line may be longer, while a sharper line might be shorter but riskier. Top racers consistently hit the optimal line, a key advantage over competitors who struggle to find it. The biggest difference between good and bad racing lines lies in how they handle the slowest part of the track.

There are two approaches to racing lines: minimizing curvature or minimizing time. The minimum curvature strategy often leads to the fastest time as well, because it allows for the highest cornering speeds without exceeding the car's limits.

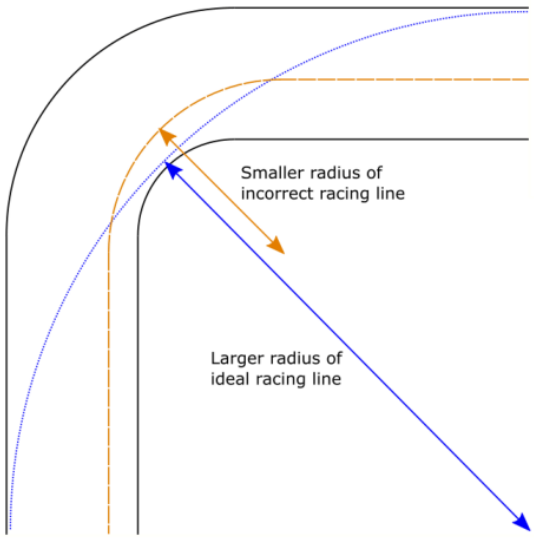

Figure 1: Comparison of racing lines

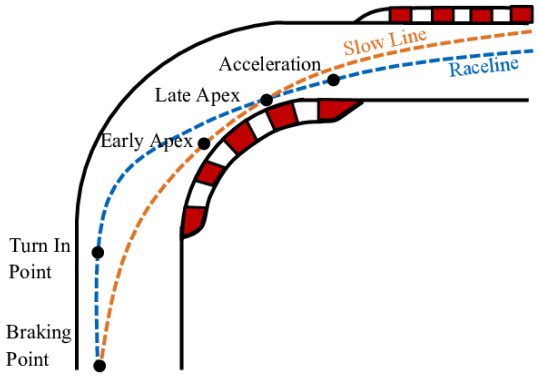

Figure 2: Optimized racing line based on constraints

2 Theory

2.1 Radius, Curvature and Speed

Suppose a_n is the centripetal acceleration, and $a_n = \frac{v^2}{r}$. When a_n is a fixed number, we have that v^2 is proportional to r, i.e., $v^2 \propto \frac{1}{r}$, where v is the maximum speed allowed and r is the radius of the corner. When r increases, v will increase. The larger r is, the less control it has over the speed.

When r is infinitely large, the corner becomes a straight line, and the maximum speed allowed will just be the physical limit of the car $v_{\max}$. In contrast, consider the curvature k. When k increases, the maximum speed allowed decreases.

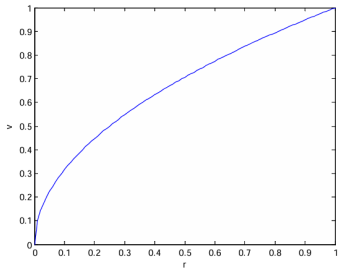

Figure 3: Relationship between the maximum allowable speed v and the radius of the corner r

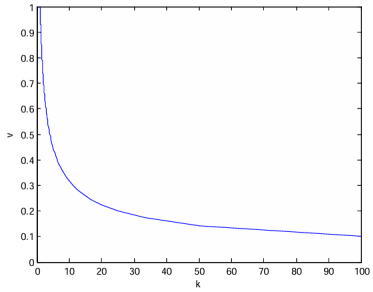

Figure 4: Relationship between the maximum allowable speed v and the curvature of the track k.

3 Problem Formulation

The objective is to minimize the total time taken for the car to complete the racing track. Mathematically, this can be represented as:

$$t = \int_0^s \frac{ds}{v}$$

where t is the total time, s is the distance traveled, and v is the velocity of the car.

Minimizing t involves finding the optimal path while considering constraints related to the track conditions and car features. These constraints shape the optimization problem for a 2-dimensional racing track.

3.1 Two Dimensional Tracks

For a two-dimensional racing track, several constraints govern the car's movement:

Friction Constraint: To ensure non-skidding, the car's velocity during turns must not exceed the threshold determined by the friction force:

$$\frac{v^2}{r} - \mu g \leq 0$$

where m is the car's mass, μ is the friction coefficient, and g is gravity's acceleration.

Turning Angle Constraint: The car must cover a total turning angle $\Delta\theta$, which can be expressed as the integral of the reciprocal of the radius of curvature:

$$\int_1^r \frac{ds}{r} = \Delta\theta$$

Turning Radius Constraint: The car's turning radius cannot be too small:

$$r_{\min} \leq r \leq r_{\max}$$

Speed Limit Constraint: The car's velocity is bounded by a maximum speed limit $v_{\max}$:

$$v \leq v_{\max}$$

Acceleration Constraint: The car's acceleration a must fall within specified bounds:

$$a_{\min} \leq a \leq a_{\max}$$

where $a_{\min}$ is the maximum deceleration and $a_{\max}$ is the maximum acceleration. Typically, $a_{\min} = -4g$ and $a_{\max} = 1.45g$, ensuring greater safety during deceleration.

The optimization problem aims to minimize the total time cost t while satisfying these constraints:

$$\text{Minimize} \quad \int_0^s \frac{1}{v} ds$$

Subject to:

$$kv^2 - \mu g \leq 0$$

$$\int_0^s k ds = \Delta\theta$$

$$r_{\min} \leq r \leq r_{\max}$$

$$v \leq v_{\max}$$

$$a_{\min} \leq a \leq a_{\max}$$

3.2 Three Dimensional Tracks

To model the full three-dimensional dynamics of the car, we consider additional forces including gravity, support force, lateral friction, and engine pull force. The total force acting on the car is represented by:

$$\vec{F}_{\text{car}} = \vec{F}_G + \vec{F}_{\text{support}} + \vec{F}_f + \vec{F}_{\text{engine}}$$

Considering the car's motion in the plane of the road, perpendicular to the direction of motion, we can express the forces as follows:

$$\vec{F}_{\text{car}} = m\vec{a}$$

Where: $\vec{F}_G$ is the force due to gravity. $\vec{F}_{\text{support}}$ is the support force. $\vec{F}_f$ is the friction force. $\vec{F}_{\text{engine}}$ is the force from the car's engine. m is the mass of the car. $\vec{a}$ is the acceleration of the car.

We introduce the following constraints:

Friction Constraint:

$$kv^2 - \mu g \cos\theta \leq 0$$

Acceleration Constraint:

$$a_{\min} \leq a \leq a_{\max}$$

Where θ is the angle between the direction of motion and the normal direction to the road. μ is the friction coefficient. g is the acceleration due to gravity.

Additionally, we consider the following factors:

a) Rolling Friction:

$$f_{\text{roll}}$$

b) Drag Force:

$$\vec{F}_{\text{drag}} = \frac{1}{2}\rho v^2 C_d A$$

Where

ρ is the density of air. v is the velocity of the car relative to the air. C_d is the drag coefficient. A is the reference area.

c) Downforce and Lift Force:

$$\vec{F}_{\text{down}} = \frac{1}{2}\rho v^2 C_L A$$

Where C_L is the lift coefficient.

The consideration of these forces and factors allows for a more comprehensive modeling of the car's dynamics and can significantly impact racing strategy. The total force acting on the car is given by:

$$\vec{F}_{\text{car}} = \vec{F}_G + \vec{F}_N + \vec{F}_f + \vec{F}_{\text{engine}} + \vec{F}_{\text{dr}} + \vec{F}_{\text{roll}} + \vec{F}_L$$

Where:

$\vec{F}_G$: Gravitational force on the car, pointing toward the ground.

$\vec{F}_N$: Normal force from the road, perpendicular to the racing track.

$\vec{F}_f$: Friction force, parallel to the track but orthogonal to the car's motion.

$\vec{F}_{\text{engine}}$: Pulling force of the car, assumed parallel to the car's motion.

$\vec{F}_{\text{dr}}$: Drag force of the car, opposing the relative motion through the air.

$\vec{F}_{\text{roll}}$: Rolling friction, opposing the motion of the car.

$\vec{F}_L$: Downforce and lift force, perpendicular to the racing track.

After further analysis, the constraints are:

$$(F_G - F_n - F_L) \cdot \mathbf{n} \geq 0$$

$$k_{\min} \leq k \leq k_{\max}$$

$$v_{\min} \leq v \leq v_{\max}$$

$$\mu_{\min} \leq \frac{F_f}{F_{\text{car}}} \leq \mu_{\max}$$

$$m_{\text{a}} \cdot a_{\min} \leq F_{\text{t}} \leq m_{\text{a}} \cdot a_{\max}$$

4 Solving the 2D Problem Using Existing Solvers

SLSQP, which stands for Sequential Least Squares Quadratic Programming, is an optimization algorithm commonly used for solving nonlinear constrained optimization problems. It belongs to the class of Sequential Quadratic Programming (SQP) methods, which iteratively solve a sequence of quadratic subproblems to approximate the solution of the original nonlinear problem.

SLSQP is implemented in various optimization libraries, including SciPy for Python.

Using SciPy-

Visit for code : https://colab.research.google.com/drive/18OFhTd3UX-1t3SRFMAh3PCV1L-4gOXj-?usp=sharing

```
import numpy as np
from scipy.optimize import minimize

# Define constants
g = 9.81
mu = 0.8
track_length = 1000
delta_theta_max = 1.0
r_max_track = 20

# Define the objective function (minimize time with penalty for large
    radius-for ease)
def objective_function(x):
    return track_length / x[1] + x[0] / x[1]**2

# Define the constraint functions
def friction_constraint(x):
    return (x[1]**2 / x[0]) - mu * g

def turning_angle_constraint(x):
    return delta_theta_max - np.arctan(1 / x[0])

def track_radius_constraint(x):
```

```
    return r_max_track - x[0]

def engine_power_constraint(x):
    max_velocity = 120  # m/s
    return max_velocity - x[1]

# Initial guess
x0 = np.array([15, 90])


bounds = [(10, 20), (70, 100)]  # bounds for radius and velocity

# Define all constraints
constraints = [
    {'type': 'ineq', 'fun': friction_constraint},
    {'type': 'ineq', 'fun': turning_angle_constraint},
    {'type': 'ineq', 'fun': track_radius_constraint},
    {'type': 'ineq', 'fun': engine_power_constraint},
]

# Solve the optimization problem
result = minimize(objective_function, x0, method='SLSQP', bounds=bounds,
    constraints=constraints)

# Print the optimal solution
print("Optimal solution:")
print("Radius:", result.x[0])
print("Velocity:", result.x[1])
print("Optimal objective value (time):", result.fun)

# Calculate additional performance metrics (assuming constant acceleration)
acceleration = (result.x[1]**2) / result.x[0] - mu * g
mass = 1000
cornering_force = mass * acceleration
```

```
56
57 print("Maximum lateral acceleration:", cornering_force / (result.x[0] *
       mass))
```

Listing 1: Python Code

```
Optimal solution:
Radius: 14.989937423532602
Velocity: 100.0
Optimal objective value (time): 10.001498993742354
Maximum lateral acceleration: 43.980583365870076
```

Figure 5: SLSQP Results

5 Alternative Implementation using PSO - ML Model

5.1 Code

```
1  # -*- coding: utf-8 -*-
2  """pso.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7      https://colab.research.google.com/drive/1
       n8K1Rn5UXgcLAYjHPl9HY4b68DrVYGRk
8  """
9
10 import math
11 import matplotlib.pyplot as plt
12
13 def plot_lines(lines):
14     for line in lines:
15         x, y = line.xy
16         plt.plot(x, y)
17
18 def get_closest_points(point, points):
```

```
    closest_point = min(points, key=lambda p: math.dist(point, p))
    return closest_point

import random
import time

class Particle:
    def __init__(self, n_dimensions, boundaries):
        self.position = [random.uniform(0, bound) for bound in boundaries]
        self.best_position = self.position[:]
        self.velocity = [random.uniform(-bound, bound) for bound in
    boundaries]

    def update_position(self, new_val):
        self.position = new_val

    def update_best_position(self, new_val):
        self.best_position = new_val

    def update_velocity(self, new_val):
        self.velocity = new_val

def optimize(cost_func, n_dimensions, boundaries, n_particles, n_iterations
    , w, cp, cg, verbose=False):
    particles = [Particle(n_dimensions, boundaries) for _ in range(
    n_particles)]
    global_solution = particles[0].position[:]
    gs_eval = cost_func(global_solution)
    gs_history = [global_solution]
    gs_eval_history = [gs_eval]

    if verbose:
        print("\n------------------ PARAMETERS -----------------")
        print(f"Number of dimensions: {n_dimensions}")
```

```
        print(f"Number of iterations: {n_iterations}")
        print(f"Number of particles: {n_particles}")
        print(f"w: {w}\tcp: {cp}\tcg: {cg}\n")
        print("----------------- OPTIMIZATION ----------------")
        print("Population initialization...")

    for p in particles:
        p_eval = cost_func(p.best_position)
        if p_eval < gs_eval:
            global_solution = p.best_position[:]
            gs_eval = p_eval

    if verbose:
        print("Start of optimization...")
        printProgressBar(0, n_iterations, prefix="Progress:", suffix="
    Complete", length=50)

    start_time = time.time_ns()

    for k in range(n_iterations):
        for p in particles:
            rp = random.uniform(0, 1)
            rg = random.uniform(0, 1)

            velocity = []
            new_position = []
            for i in range(n_dimensions):
                vel_component = (
                    w * p.velocity[i] +
                    cp * rp * (p.best_position[i] - p.position[i]) +
                    cg * rg * (global_solution[i] - p.position[i])
                )

                vel_component = max(-boundaries[i], min(vel_component,
```

```
    boundaries[i]))
83                 velocity.append(vel_component)
84
85                 new_pos_component = p.position[i] + velocity[i]
86                 new_pos_component = max(0.0, min(new_pos_component,
    boundaries[i]))
87                 new_position.append(new_pos_component)
88
89             p.update_velocity(velocity)
90             p.update_position(new_position)
91
92             p_eval = cost_func(p.position)
93             if p_eval < cost_func(p.best_position):
94                 p.update_best_position(p.position)
95                 if p_eval < gs_eval:
96                     global_solution = p.position[:]
97                     gs_eval = p_eval
98
99         gs_eval_history.append(gs_eval)
100        gs_history.append(global_solution)
101
102        if verbose:
103            printProgressBar(k + 1, n_iterations, prefix="Progress:",
    suffix="Complete", length=50)
104
105    finish_time = time.time_ns()
106    elapsed_time = (finish_time - start_time) / 10e8
107
108    if verbose:
109        time.sleep(0.2)
110        print("\nEnd of optimization...\n")
111        print("------------------- RESULTS --------------------")
112        print(f"Optimization elapsed time: {elapsed_time:.2f} s")
113        print(f"Solution evaluation: {gs_eval:.5f}")
```

```
114
115     return global_solution, gs_eval, gs_history, gs_eval_history
116
117 def printProgressBar(iteration, total, prefix='', suffix='', decimals=1,
    length=100, fill='    ', printEnd="\r"):
118     percent = ("{0:." + str(decimals) + "f}").format(100 * (iteration /
    float(total)))
119     filledLength = int(length * iteration // total)
120     bar = fill * filledLength + '-' * (length - filledLength)
121     print(f'\r{prefix} |{bar}| {percent}% {suffix}', end=printEnd)
122     if iteration == total:
123         print()
124
125 import json
126 import math
127 import matplotlib.pyplot as plt
128 import numpy as np
129
130
131 from scipy import interpolate
132 from shapely.geometry import LineString
133
134
135 def main():
136   # PARAMETERS
137   N_SECTORS = 30
138   N_PARTICLES = 65
139   N_ITERATIONS = 150
140   W = -0.2264
141   CP = -0.1556
142   CG = 3.8879
143   PLOT = True
144
145   # Read tracks from json file
```

```
146 json_data = None
147 with open('tracks.json') as file:
148   json_data = json.load(file)
149
150 track_layout = json_data['test_track']['layout']
151 track_width = json_data['test_track']['width']
152
153 # Compute inner and outer tracks borders
154 center_line = LineString(track_layout)
155 inside_line = LineString(center_line.parallel_offset(track_width/2, 'left
      '))
156 outside_line = LineString(center_line.parallel_offset(track_width/2, '
      right'))
157
158 if PLOT:
159   plt.title("Track Layout Points")
160   for p in track_layout:
161     plt.plot(p[0], p[1], 'r.')
162   plt.show()
163   plt.title("Track layout")
164   plot_lines([outside_line, inside_line])
165   plt.show()
166
167 # Define sectors' extreme points (in coordinates). Each sector segment is
       defined by an inside point and an outside point.
168 inside_points, outside_points = define_sectors(center_line, inside_line,
      outside_line, N_SECTORS)
169
170 if PLOT:
171   plt.title("Sectors")
172   for i in range(N_SECTORS):
173     plt.plot([inside_points[i][0], outside_points[i][0]], [inside_points[
      i][1], outside_points[i][1]])
174   plot_lines([outside_line, inside_line])
```

```
175     plt.show()
176
177 # Define the boudaries that will be passed to the pso algorithm [0 -
      trackwidth].
178 # 0 correspond to the inner border, trackwidth to the outer border.
179 boundaries = []
180 for i in range(N_SECTORS):
181     boundaries.append(np.linalg.norm(inside_points[i]-outside_points[i]))
182
183 def myCostFunc(sectors):
184     return get_lap_time(sectors_to_racing_line(sectors, inside_points,
      outside_points))
185
186 global_solution, gs_eval, gs_history, gs_eval_history = optimize(
      cost_func = myCostFunc,n_dimensions=N_SECTORS, boundaries=boundaries,
      n_particles=N_PARTICLES,  n_iterations=N_ITERATIONS,  w=W, cp=CP, cg=CG,
          verbose=True)_, v, x, y = get_lap_time(sectors_to_racing_line(
      global_solution, inside_points, outside_points), return_all=True)
187
188 if PLOT:
189     plt.title("Racing Line History")
190     plt.ion()
191     for i in range(0, len(np.array(gs_history)), int(N_ITERATIONS/100)):
192         lth, vh, xh, yh = get_lap_time(sectors_to_racing_line(gs_history[i],
      inside_points, outside_points), return_all=True)
193         plt.scatter(xh, yh, marker='.', c = vh, cmap='RdYlGn')
194         plot_lines([outside_line, inside_line])
195         plt.draw()
196         plt.pause(0.00000001)
197         plt.clf()
198     plt.ioff()
199
200     plt.title("Final racing line")
201     rl = np.array(sectors_to_racing_line(global_solution, inside_points,
```

```
    outside_points))
202     plt.plot(rl[:,0], rl[:,1], 'ro')
203     plt.scatter(x, y, marker='.', c = v, cmap='RdYlGn')
204     for i in range(N_SECTORS):
205       plt.plot([inside_points[i][0], outside_points[i][0]], [inside_points[
    i][1], outside_points[i][1]])
206     plot_lines([outside_line, inside_line])
207     plt.show()
208 
209     plt.title("Global solution history")
210     plt.ylabel("lap_time (s)")
211     plt.xlabel("n_iteration")
212     plt.plot(gs_eval_history)
213     plt.show()
214 
215 def sectors_to_racing_line(sectors:list, inside_points:list, outside_points
    :list):
216   racing_line = []
217   for i in range(len(sectors)):
218     x1, y1 = inside_points[i][0], inside_points[i][1]
219     x2, y2 = outside_points[i][0], outside_points[i][1]
220     m = (y2-y1)/(x2-x1)
221 
222     a = math.cos(math.atan(m)) # angle with x axis
223     b = math.sin(math.atan(m)) # angle with x axis
224 
225     xp = x1 - sectors[i]*a
226     yp = y1 - sectors[i]*b
227 
228     if abs(math.dist(inside_points[i], [xp,yp])) + abs(math.dist(
    outside_points[i], [xp,yp])) - \
229         abs(math.dist(outside_points[i], inside_points[i])) > 0.1:
230       xp = x1 + sectors[i]*a
231       yp = y1 + sectors[i]*b
```

```
232
233     racing_line.append([xp, yp])
234   return racing_line
235
236 def get_lap_time(racing_line:list, return_all=False):
237   # Computing the spline function passing throught the racing_line points
238   rl = np.array(racing_line)
239   tck, _ = interpolate.splprep([rl[:,0], rl[:,1]], s=0.0, per=0)
240   x, y = interpolate.splev(np.linspace(0, 1, 1000), tck)
241
242   # Computing derivatives
243   dx, dy = np.gradient(x), np.gradient(y)
244   d2x, d2y = np.gradient(dx), np.gradient(dy)
245
246   curvature = np.abs(dx * d2y - d2x * dy) / (dx * dx + dy * dy)**1.5
247   radius = [1/c for c in curvature]
248
249   us = 0.13   # Coefficient of friction
250
251   # Computing speed at each point
252   v = [min(10,math.sqrt(us*r*9.81)) for r in radius]
253
254   # Computing lap time around the track
255   lap_time = sum( [math.sqrt((x[i]-x[i+1])**2 + (y[i]-y[i+1])**2)/v[i] for
      i in range(len(x)-1)])
256
257   if return_all:
258     return lap_time, v, x, y
259   return lap_time
260
261 def define_sectors(center_line : LineString, inside_line : LineString,
     outside_line : LineString, n_sectors : int):
262   distances = np.linspace(0, center_line.length, n_sectors)
263   center_points_temp = [center_line.interpolate(distance) for distance in
```

```
    distances]
264   center_points = np.array([[center_points_temp[i].x, center_points_temp[i
      ].y] for i in range(len(center_points_temp)-1)])
265   center_points = np.append(center_points, [center_points[0]], axis=0)
266
267   distances = np.linspace(0,inside_line.length, 1000)
268   inside_border = [inside_line.interpolate(distance) for distance in
      distances]
269   inside_border = np.array([[e.x, e.y] for e in inside_border])
270   inside_points = np.array([get_closest_points([center_points[i][0],
      center_points[i][1]], inside_border) for i in range(len(center_points))
      ])
271
272   distances = np.linspace(0,outside_line.length, 1000)
273   outside_border = [outside_line.interpolate(distance) for distance in
      distances]
274   outside_border = np.array([[e.x, e.y] for e in outside_border])
275   outside_points = np.array([get_closest_points([center_points[i][0],
      center_points[i][1]], outside_border) for i in range(len(center_points))
      ])
276
277   return inside_points, outside_points
278
279 if __name__ == "__main__":
280   main()
```

Listing 2: Python Code

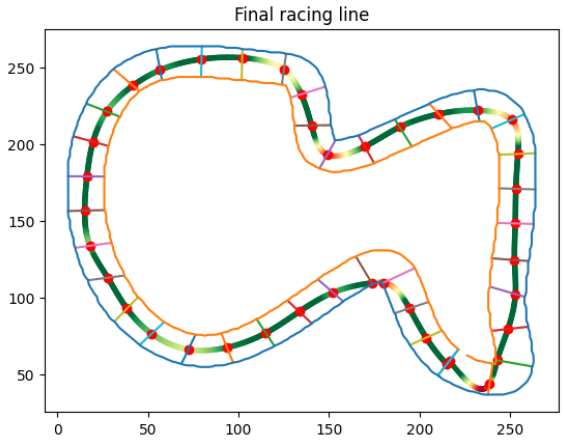

Figure 6: Final Racing Line generated by ML model

6 Conclusion

In conclusion, this document provides a comprehensive exploration of racing line optimization using both traditional optimization methods and a machine learning approach.

Firstly, the problem of racing line optimization is introduced, highlighting its significance in achieving faster lap times on racing tracks. Theoretical background on radius, curvature, and speed dynamics is provided.

Next, the problem is formulated as a nonlinear constrained optimization problem, considering various track constraints such as friction, turning angle, turning radius, speed limits, and acceleration limits.

Two different approaches to solving this optimization problem are explored:

1. Traditional Optimization with SLSQP: The Sequential Least Squares Quadratic Programming (SLSQP) algorithm is utilized, implemented through SciPy in Python. Code snippets and results are provided, showcasing the optimal racing line obtained using SLSQP, along with additional performance metrics such as maximum lateral acceleration.

2. Machine Learning Approach with Particle Swarm Optimization (PSO): An alternative approach using PSO is presented, where particles are optimized to find the racing line that minimizes lap time. The output includes the final racing line generated by the PSO algorithm, demonstrating its effectiveness in optimizing racing lines.

7 References

[1] YouTube. "Dynamic programming for racing line optimization in Python." [Online Video]. Available: https://www.youtube.com/watch?v=BuqoSJFyvVE.

[2] Massachusetts Institute of Technology (MIT). "Racing Line Optimization." [Online]. Available: https://dspace.mit.edu/bitstream/handle/1721.1/64669/706825301-MIT.pdf.

[3] ScienceDirect. "Computing the racing line using Bayesian optimization." [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352146517304593/pdf?md5=d49f1e998498eb57861f888ddcd94869&pid=1-s2.0-S2352146517304593-main.pdf.

[4] Medium. "Genetic Programming for Racing Line Optimization - Part 1." [Online]. Available: https://medium.com/adventures-in-autonomous-vehicles/genetic-programming-for-racing-line-optimization-part-1-e563c606e502.